UTM · MATERIA DE COMPILADORES

Compilador

Equipo 6 — Base de integración

GUÍA DE USUARIO

Instalación, uso y solución de problemas

Aplicación PyQt6

Analizadores en Python y Flex · Módulo de gramáticas · Colección canónica LR(0)

Datos del documento	
Fecha	30 de Octubre del 2026
Integrantes	García Barela Ángel Andrés Sánchez Aguilar Luis Ángel Álvarez Olivera Irving Damián Ramírez Manzano Jaret López Osorio Haniel Martínez Cesar Zaid Soriano Martínez Miguel Ángel
Profesor	David Martínez Torres

1. Introducción

1.1 Propósito de este documento

Esta guía explica cómo instalar, ejecutar y usar el **Compilador del Equipo 6**, una aplicación de escritorio desarrollada con **PyQt6** que reúne en una sola ventana los componentes del proyecto:

- Dos **analizadores léxicos** (uno escrito en Python y otro generado con Flex).
- Un módulo de **PRIMEROS y SIGUIENTES**.
- Un módulo de **colección canónica LR(0)**.

Está pensada para cualquier persona que necesite poner en marcha el proyecto en **Windows o Linux**, sin necesidad de conocer su código interno. Incluye requisitos, pasos de instalación, descripción de la interfaz, limitaciones conocidas y una sección de solución de problemas.

1.2 Componentes de la aplicación

El programa se inicia desde un único archivo (`INSTALL/main.py`) y ofrece **cuatro pestañas de trabajo**:

Pestaña	Función	Tecnología
Léxico Python	Analiza código Pascal con el analizador léxico implementado en Python y muestra tokens, símbolos y errores.	Python
Léxico Flex	Compila el archivo `.l` del proyecto y analiza código Pascal con el binario generado.	Flex + GCC
PRIMEROS y SIGUIENTES	Carga y valida una gramática; calcula y muestra los conjuntos PRIMEROS y SIGUIENTES.	Python
Colección canónica	Carga y valida una gramática; calcula y muestra la colección canónica LR(0) (estados, transiciones y estados de aceptación).	Python
Tabla de análisis sintáctico	Carga y valida una gramática; calcula y muestra la tabla de análisis sintáctico LR (estados de aceptación, reglas de desplazamiento y reducción entre reglas).	Python
Analizador sintáctico final	Carga y valida una gramática y una tira de tokens correspondiente; calcula y muestra el análisis sintáctico final (impresiones a pantalla, estados de aceptación y errores).	Python

2. Requisitos previos

La aplicación necesita **Python 3.10 o superior, PyQt6, Flex y GCC**, y todos deben estar disponibles en el mismo entorno desde el que se lanza. No es necesario instalarlos a mano: los instaladores incluidos con el proyecto (sección 4) descargan y configuran todo automáticamente.

Componente	En Windows	En Linux (Ubuntu / Debian)
Sistema	Windows 10 (versión 1809 o posterior) o Windows 11, con internet y `winget`.	Ubuntu / Debian con `apt` e internet.
Python 3.10+	Lo instala `instalar_windows.bat` si falta.	Lo instala `instalar_linux.sh`.
PyQt6	Se instala con `requirements.txt`.	Se instala con `requirements.txt`.
Componente	**En Windows**	**En Linux (Ubuntu / Debian)**
Flex y GCC	Los instala `instalar_windows.bat` mediante MSYS2.	Los instala `instalar_linux.sh`.
Librerías gráficas	No requiere pasos adicionales.	`libegl1`, `libgl1`, `libxcb-cursor0`, `libxkbcommon-x11-0` (las instala el script).

3. Estructura del proyecto

La distribución de carpetas separa claramente la **presentación**, la **lógica**, la **persistencia**, los **modelos de gramática** y los **archivos de instalación y ejecución**. Estas son las rutas más relevantes:

Carpeta / archivo	Contenido
`INSTALL/main.py`	Punto de entrada: inicia la aplicación.
`INSTALL/requirements.txt`	Dependencias de Python (incluye PyQt6).
`INSTALL/instalar_windows.bat` y `INSTALL/instalar_windows.ps1`	Instalador automático para Windows (se ejecuta una sola vez).
`INSTALL/ejecutar.bat`	Abre la aplicación en Windows.
`INSTALL/instalar_linux.sh` y `INSTALL/ejecutar_linux.sh`	Instalador y lanzador para Linux.
`INSTALL/.venv/`	Entorno virtual de Python (se genera al instalar).
`PRESENTATION/`	Pestañas PyQt6, estilos y utilidades de interfaz.
`LOGIC/python_lexer/`	Analizador léxico implementado en Python.
`LOGIC/flex_lexer/`	Analizador Flex: código `.l` fuente y puente con la GUI.
`LOGIC/first_follow/`	Algoritmos de PRIMEROS y SIGUIENTES.
`LOGIC/canonical_collection/`	Algoritmo de colección canónica LR(0).
`LOGIC/common/`	Utilidades compartidas entre capas.
`PERSISTENCE/`	Lectura y escritura de archivos (gramáticas, TSV, texto).
`GRAMMAR_MODELS/`	Clases `Grammar`, `Production` y `Symbol`.

`BUILD/flex/`	Archivos generados por Flex (código C y binario). Puede borrarse cuando no haya tareas activas.
`EXAMPLES/`	Ejemplos de entrada para las cuatro pestañas.
`TESTS/`	Pruebas automatizadas.
`DOCUMENTATION/`	Documentación adicional del proyecto.
`Legacy/`	Interfaces anteriores, conservadas solo como referencia histórica.

> **Nota importante:** los archivos `main.py`, `requirements.txt`, los instaladores y los lanzadores viven **dentro de `INSTALL/`**. El resto de carpetas (`PRESENTATION/`, `LOGIC/`, `PERSISTENCE/`, `GRAMMAR_MODELS/`, etc.) están en la **raíz del proyecto**, un nivel arriba de `INSTALL/`.

4. Instalación y ejecución

La instalación es **automática**: basta con ejecutar un archivo, que descarga e instala todo lo necesario. **Solo se hace una vez**; después la aplicación se abre con otro archivo. Seguir la sección 4.1 si se usa **Windows** o la 4.2 si se usa **Linux**.

> **Nota:** los archivos `instalar_windows.bat`, `instalar_windows.ps1`, `ejecutar.bat`, `instalar_linux.sh`, `ejecutar_linux.sh`, `main.py` y `requirements.txt` deben estar **dentro de la carpeta `INSTALL/`**.

4.1 Windows

Requisitos: Windows 10 (versión 1809 o posterior) o Windows 11, conexión a internet y poder aceptar una ventana de permisos de administrador.

Paso 1 — Abrir la carpeta del proyecto

En el Explorador de archivos, abrir la carpeta `INSTALL/`, que contiene `main.py`, `instalar_windows.bat` y `ejecutar.bat`.

Paso 2 — Instalar (solo la primera vez)

Hacer doble clic en `instalar_windows.bat`. Se abre una ventana negra que muestra el avance en **cuatro etapas**. Si Windows pide permiso para instalar programas, pulsar **Sí**. El proceso tarda varios minutos y no hay que escribir nada.

Etapa	Qué hace
1. Python	Si no hay Python 3.10 o superior, lo descarga e instala (versión 3.12) con `winget`.
2. MSYS2	Instala MSYS2 en `C:\msys64` y, con él, Flex y GCC.
3. Entorno virtual	Crea la carpeta `.venv` e instala PyQt6 y las demás dependencias de `requirements.txt`.
4. Verificación	Comprueba que Flex, GCC y PyQt6 funcionan.

Al final aparece el mensaje **"Instalación terminada"**. Si en cambio aparece un mensaje en rojo, ver la sección 9.

Paso 3 — Abrir la aplicación

Hacer doble clic en `ejecutar.bat`. Este archivo prepara Flex y GCC automáticamente cada vez que se abre, por lo que no hace falta modificar el `PATH` ni abrir terminales. Para volver a usar el programa en otra ocasión, basta con repetir solo este paso.

> **Nota:** si Windows muestra *"Windows protegió su PC"*, pulsar **Más información** y luego **Ejecutar de todas formas**: ocurre con archivos `.bat` descargados.

> **Importante:** esta ruta no ha sido certificada con todas las distribuciones de Flex para Windows. Si el instalador automático falla, se puede instalar a mano siguiendo el **Anexo A**.

4.2 Linux (Ubuntu / Debian)

Paso 1 — Abrir una terminal en la raíz del proyecto

Abrir la carpeta raíz del proyecto (la que contiene `INSTALL/`, `PRESENTATION/`, `LOGIC/`, etc.) y, con clic derecho, elegir **Abrir en terminal** (o usar `cd` hasta esa carpeta).

Paso 2 — Instalar (solo la primera vez)

```
chmod +x INSTALL/instalar_linux.sh INSTALL/ejecutar_linux.sh
./INSTALL/instalar_linux.sh
```

Paso 3 — Abrir la aplicación

```
./INSTALL/ejecutar_linux.sh
```

Instalación manual (alternativa)

Si se prefiere no usar los scripts, estos son los comandos equivalentes (desde la raíz del proyecto):

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip flex gcc \
    libegl1 libgl1 libxcb-cursor0 libxkbcommon-x11-0 && \
    cd INSTALL && python3 -m venv .venv && \
    .venv/bin/python -m pip install -r requirements.txt && \
    .venv/bin/python main.py
```

> **Nota:** en otras distribuciones de Linux, instalar los paquetes equivalentes (Python 3 con venv y pip, Flex, GCC y las librerías gráficas de Qt) con el gestor de paquetes correspondiente.

5. Descripción de la interfaz

La ventana principal está organizada en **cuatro pestañas**. Aunque cada una tiene su propio propósito, comparten los mismos elementos básicos:

Elemento	Descripción
Pestañas superiores	Permiten cambiar entre *Léxico Python*, *Léxico Flex*, *PRIMEROS y SIGUIENTES* y *Colección canónica*.
Editor de texto	Área donde se escribe o abre el código Pascal (o la gramática). Cada pestaña conserva su propio texto.
Botón Analizar	Ejecuta el análisis sobre el texto actual del editor.
Botón Compilar Flex	Solo en la pestaña Flex: genera el analizador a partir del archivo `.l`.
Botón Cargar y validar gramática	Solo en las pestañas de PRIMEROS/SIGUIENTES y Colección canónica: lee la gramática y valida su formato.
Guardar como…	Guarda el contenido del editor en un archivo. Es una acción explícita.
Selector inferior	Elige qué tabla se muestra (Tokens, Símbolos, Errores, PRIMEROS, SIGUIENTES, Estados, Transiciones).
Guardar tabla TSV	Exporta únicamente la tabla seleccionada en formato TSV (valores separados por tabulaciones).

6. Uso de la aplicación

6.1 Analizador Python

1. Abrir la pestaña **Léxico Python**.
2. Abrir un ejemplo de `EXAMPLES/pythonLexer/` o escribir código Pascal directamente en el editor.
3. Pulsar **Analizar**.
4. Revisar los resultados con el selector inferior (**Tokens**, **Símbolos**, **Errores**).

6.2 Analizador Flex

1. Abrir la pestaña **Léxico Flex** y pulsar **Compilar Flex**.
2. Abrir un ejemplo de `EXAMPLES/flexLexer/` o escribir código Pascal en el editor.
3. Pulsar **Analizar**.
4. Revisar los resultados con el selector inferior.

El archivo `.l` del proyecto se selecciona por defecto. Para modificarlo, editarlo en el IDE y volver a compilar: la aplicación detecta si el `.l` cambió desde la última compilación.

> **Nota:** las advertencias `yyunput defined but not used` e `input defined but not used` que emite GCC al compilar son **habituales** del código generado por Flex y no impiden usar el analizador.

Flex genera el código C y el binario dentro de `BUILD/flex/`. Los archivos TSV temporales se eliminan automáticamente al terminar cada análisis.

> **Advertencia:** el botón **Seleccionar .l** permite cambiar el analizador Flex por otro archivo, pero **no debe usarse** salvo que se cuente con otro analizador `.l` preparado para este proyecto (que genere los tres TSV con el formato esperado). En el uso normal, dejar el archivo por defecto.

6.3 PRIMEROS y SIGUIENTES

1. Abrir la pestaña **PRIMEROS y SIGUIENTES**.
2. Pulsar **Cargar y validar gramática** y seleccionar un archivo de `EXAMPLES/first_follow/`.
3. Verificar la vista previa de la gramática (terminales, no terminales, símbolo inicial y producciones).
4. Pulsar **Calcular PRIMEROS y SIGUIENTES**.
5. Cambiar el selector inferior para ver **PRIMEROS** o **SIGUIENTES**.

6.4 Colección canónica LR(0)

1. Abrir la pestaña **Colección canónica**.
2. Pulsar **Cargar y validar gramática** y seleccionar un archivo de `EXAMPLES/canonical_collection/`.
3. Verificar la vista previa de la gramática.

4. Pulsar **Calcular colección canónica**.
5. Cambiar el selector inferior para ver **Estados** (cada estado con sus ítems LR(0)) o **Transiciones** (origen, símbolo, destino y si el destino es de aceptación).

6.5 Tabla de Análisis Sintáctico LR

1. Abrir la pestaña **Tabla de análisis LR**.
2. Pulsar **Cargar y validar gramática** y seleccionar un archivo de `EXAMPLES/canonical_collection/`.
3. Verificar la vista previa de la gramática (terminales, no terminales, símbolo inicial y producciones).
4. Pulsar **Construir tabla LR**.
5. Guardar el resultado final pulsando en **Guardar tabla TSV**.

6.6 Analizador Sintáctico Final LR

1. Abrir la pestaña **Analizador sintáctico LR**.
2. Pulsar **Cargar gramática y L.Tokens** y seleccionar un archivo de `EXAMPLES/lr_parser/`.
3. Verificar la vista previa de la gramática (terminales, no terminales, símbolo inicial y producciones) y de la tira de tokens en la parte inferior de la ventana.
4. Pulsar **Analizar sintácticamente**.
5. Guardar el resultado final pulsando en **Guardar tabla TSV**.

La gramática aumentada debe terminar su primera producción con el terminal `$`, que marca el fin de la entrada. La aplicación reconoce `$` como terminal válido.

6.6 Tablas de resultados y exportación

Tabla	Contenido
Tokens	Elementos léxicos reconocidos en el texto analizado.
Símbolos	Tabla de símbolos. En el analizador Python lista las variables declaradas; en Flex lista los identificadores que reconoce.
Errores	Problemas detectados durante el análisis (por ejemplo, caracteres no válidos o límites excedidos).
PRIMEROS	Conjuntos PRIMEROS de cada no terminal.
SIGUIENTES	Conjuntos SIGUIENTES de cada no terminal.
Estados	Estados de la colección canónica LR(0), con sus ítems.
Transiciones	Transiciones entre estados, símbolo que las provoca y marca de aceptación.

Guardar tabla TSV exporta solo la tabla que esté seleccionada en ese momento. El TSV puede abrirse con Excel, LibreOffice Calc o cualquier editor de texto.

6.7 Consideraciones importantes

6.7.1 Analizar usa el texto del editor y no modifica el archivo Pascal original. Para guardar cambios hay que usar **Guardar como…**.

6.7.2 Cada pestaña conserva su propio texto de forma independiente.

6.7.3 Los análisis y compilaciones se ejecutan en segundo plano, sin congelar la ventana. Mientras haya una operación en curso, la aplicación no permite cerrar la ventana hasta que termine.

6.7.4 La carpeta `BUILD/flex/` puede borrarse cuando no haya tareas activas; se vuelve a generar al compilar.

7 Pruebas

Para ejecutar las pruebas automatizadas, abrir una terminal en la raíz del proyecto.

En Windows (PowerShell):

```
$env:Path = "C:\msys64\ucrt64\bin;" + $env:Path
INSTALL\.venv\Scripts\python.exe -m unittest discover -s TESTS -v
```

En Linux:

```
INSTALL/.venv/bin/python -m unittest discover -s TESTS -v
```

- Las pruebas gráficas usan Qt *offscreen*, por lo que no abren ventanas.
- Las pruebas de Flex se omiten con un aviso si `flex` y `gcc` no se encuentran en el `PATH`.
- Las pruebas de SIGUIENTES se comparan contra conjuntos PRIMEROS esperados definidos de forma explícita, no contra el cálculo temporal de PRIMEROS del módulo de gramáticas.

8 Limitaciones conocidas

Los dos analizadores léxicos son entregas independientes: **no se unificó su catálogo de tokens ni su criterio de tabla de símbolos**, por lo que sus resultados pueden diferir para un mismo código.

Elemento	Comportamiento
Tabla de símbolos (Python)	Lista las variables declaradas.
Tabla de símbolos (Flex)	Lista los identificadores que reconoce.
Arreglos de Flex	Máximo de 5000 elementos.
Longitud de lexema (Flex)	Máximo de 255 bytes.
Exceso de límite	Se informa un error; no se escribe fuera del arreglo.
TSV de Flex	Tabulaciones, saltos de línea y barras invertidas se representan como secuencias escapadas.
Nombres de token	Flex y Python usan nombres distintos para algunos tokens (por ejemplo `Punto_Coma` vs `SEMICOLON`). Es una decisión de diseño actual; la unificación está prevista para etapas posteriores.
Flex en Windows	Se obtiene con MSYS2 (lo instala `instalar_windows.bat`); no está certificado con todas las distribuciones de Flex para Windows.

9 Solución de problemas

Problema	Causa probable	Solución
El instalador dice que no encuentra `winget`	Windows sin "Instalador de aplicaciones" (App Installer).	Instalarlo desde Microsoft Store o seguir la instalación manual del **Anexo A**.
El instalador dice que Python aún no está disponible	Windows no ha actualizado el `PATH` de la ventana actual.	Cerrar la ventana y volver a ejecutar `instalar_windows.bat`.

Problema	Causa probable	Solución
El instalador dice que MSYS2 no quedó en `C:\msys64`	La instalación falló o se hizo en otra ruta.	Instalar `MSYS2.exe` desde `msys2.org` en la ruta por defecto y volver a ejecutar el instalador.
Windows muestra "Windows protegió su PC"	Archivo `.bat` sin firma digital.	Pulsar **Más información** y luego **Ejecutar de todas formas**.
`ejecutar.bat` dice "Primero ejecuta instalar_windows.bat"	Aún no existe el entorno virtual `.venv`.	Ejecutar `instalar_windows.bat` y esperar a que termine.
Error `No module named PyQt6`	La aplicación se abrió fuera del entorno virtual.	Abrirla siempre con `ejecutar.bat` (Windows) o `./INSTALL/ejecutar_linux.sh` (Linux); si persiste, repetir la instalación.
`Could not open requirements file`	`requirements.txt` no está dentro de `INSTALL/`.	Verificar que exista `INSTALL/requirements.txt` con la línea `PyQt6`.
La pestaña Flex no compila o no encuentra `flex` / `gcc`	La aplicación se abrió sin `ejecutar.bat` (Windows) o Flex/GCC no se instalaron.	Abrir con `ejecutar.bat`; si persiste, volver a ejecutar el instalador.
Flex falla indicando que no encuentra `m4` (Windows)	Falta `m4` en MSYS2.	Abrir MSYS2 UCRT64 y ejecutar `pacman -S m4`.
Error del plugin de plataforma Qt (`xcb`) en Linux	Faltan librerías gráficas del sistema.	Volver a ejecutar `./INSTALL/instalar_linux.sh`, que instala `libegl1`, `libgl1`, `libxcb-cursor0` y `libxkbcommon-x11-0`.
`Permission denied` al ejecutar un `.sh` (Linux)	El script no tiene permiso de ejecución.	Ejecutar `chmod +x INSTALL/instalar_linux.sh INSTALL/ejecutar_linux.sh`.
El análisis Flex muestra resultados de un `.l` anterior	El `.l` se modificó y no se recompiló.	Pulsar de nuevo **Compilar Flex** y volver a **Analizar**.
Error por límite excedido en Flex	Más de 5000 elementos o lexema mayor a 255 bytes.	Dividir la entrada o acortar el lexema que supera el límite.
No se puede cerrar la ventana	Hay un análisis o compilación en curso.	Esperar a que la operación termine.
Pruebas de Flex omitidas	`flex` o `gcc` no están en el `PATH`.	Seguir los comandos de la sección 7, que preparan el `PATH` antes de ejecutar las pruebas.
La gramática aumentada no carga con `$`	Versión anterior del lector rechazaba `$` como terminal.	Actualizar `PERSISTENCE/grammar_reader.py` a la versión que permite `$` como terminal.
Advertencias `yyunput defined but not used` e `input defined but not used`	Código auxiliar generado por Flex que no se usa.	Son **advertencias normales**. No impiden usar el analizador; se pueden ignorar.

10 Documentación adicional

- `DOCUMENTATION/VALIDACION.md` : comprobaciones realizadas y sus límites.
- `DOCUMENTATION/First_Follow/` : documentación de PRIMEROS y SIGUIENTES.
- `DOCUMENTATION/Canonical_Collection/` : documentación de la colección canónica.
- `DOCUMENTATION/FlexLexer/` y `DOCUMENTATION/PythonLexer/` : documentación de cada analizador léxico.
- `Legacy/`: interfaces anteriores, solo como referencia histórica. No son entradas de la aplicación actual; no deben importarse ni ejecutarse esperando que funcionen con la distribución nueva.

11 Asignaciones del equipo

11.6 Etapa 1 — Programa fuente, alfabeto, AFD y errores

No.	Actividad	Descripción	Responsable(s)
1	Programa de ejemplo en Pascal	Elaborar un programa fuente con bloque principal, funciones o procedimientos, variables simples, arreglos, apuntadores simples y dobles, tipos primitivos, entrada/salida y estructuras de control.	Ramírez Manzano Jaret
2	Alfabeto y expresiones regulares	Analizar el programa de ejemplo, identificar letras, dígitos, operadores, delimitadores, palabras reservadas, identificadores, números, literales, comentarios y espacios. Definir las expresiones regulares.	Sánchez Aguilar Luis Ángel · Álvarez Olivera Irving Damián
3	Autómata finito determinista	Diseñar el AFD que reconozca los componentes léxicos definidos: estados, transiciones, tabla de estados finales y token asociado.	López Osorio Haniel · Martínez Cesar Zaid
4	Tabla de errores	Definir qué caracteres serán no válidos para el lenguaje.	García Barela Ángel Andrés · Soriano Martínez Miguel Ángel

11.7 Etapa 2 — Implementación del analizador y GUI

No.	Actividad	Descripción	Responsable(s)
1	Desarrollo de la interfaz gráfica	Ventana principal, editor con números de línea y ventanas de resultados. Apertura de archivos, análisis y limpieza. Conexión de la interfaz con el analizador.	Ramírez Manzano Jaret

No.	Actividad	Descripción	Responsable(s)
2	Gestión de errores léxicos y tabla de errores	Definir errores a detectar y su recuperación. Registro de caracteres inválidos, cadenas y comentarios sin cerrar. Entrega de datos a la interfaz.	López Osorio Haniel
3	Análisis previo y diseño del sistema	Requisitos del analizador, diagrama UML, responsabilidades de los módulos y planificación.	Sánchez Aguilar Luis Ángel
4	Gestión de la tabla de símbolos	Extracción de nombres de variables a partir de los tokens. Evitar nombres repetidos.	Martínez Cesar Zaid
5	Lectura del código y reconocimiento de lexemas	Recorrido carácter por carácter, control de posición, línea y columna. Delimitación de identificadores, números, cadenas y operadores. Omisión de espacios y comentarios.	García Barela Ángel Andrés
6	Definición y clasificación de tokens	Estructura de cada token y su catálogo. Diccionarios de palabras reservadas y símbolos. Clasificación de lexemas.	Soriano Martínez Miguel Ángel
7	Integración, pruebas y preparación de la entrega	Integración del lexer, tabla de símbolos y errores. Pruebas de funcionamiento conjunto.	Álvarez Olivera Irving Damián

11.8 Etapa 3 — PRIMEROS y SIGUIENTES

No.	Actividad	Descripción	Responsable(s)
1	Coordinación e integración	Organizar actividades, integrar lectura de gramáticas, algoritmos e interfaz.	Sánchez Aguilar Luis Ángel
2	Lectura y validación de gramáticas	Leer terminales, no terminales, símbolo inicial y producciones. Reconocer `id`, `V'`, `ε`. Detectar datos faltantes o inconsistentes.	García Barea Ángel Andrés
3	Algoritmo de PRIMEROS	Cálculo de PRIMEROS considerando producciones vacías, cadenas de símbolos y recursión.	López Osorio Haniel
4	Algoritmo de SIGUIENTES	Cálculo de SIGUIENTES a partir de PRIMEROS. Incorporar `$` al símbolo inicial y propagar hasta punto fijo.	Álvarez Olivera Irving Damián
5	Interfaz gráfica	Ventana para cargar archivos, visualizar la gramática, ejecutar el cálculo y mostrar PRIMEROS, SIGUIENTES y errores.	Ramírez Manzano Jaret
6	Pruebas y verificación	Cálculo manual de resultados esperados y comparación con el programa.	Soriano Martínez Miguel Ángel
7	Arquitectura, UML e informe	Organización de clases, diagrama de clases, informe con índice y evidencias.	Martínez Cesar Zaid

11.9 Etapa 4 — Colección canónica LR(0)

No.	Actividad	Descripción	Responsable
1	Colección canónica	Implementar el algoritmo que construye el estado inicial, recorre estados y símbolos, utiliza `Ir_a` y agrega únicamente estados nuevos.	García Barea Ángel Andrés
2	Lectura y validación de la gramática	Cargar el archivo y obtener terminales, no terminales, símbolo inicial y producciones. Validar componentes y la producción aumentada.	García Barea Ángel Andrés
3	Algoritmo Cerradura	Implementar `cerradura(I)`, agregando los elementos necesarios hasta que no aparezcan nuevos. Manejar recursión y producciones vacías.	Álvarez Olivera Irving Damián
4	Algoritmo Ir_a y comparación de estados	Implementar `ir_a(I, X)`: desplazar el punto sobre el símbolo indicado y aplicar Cerradura. Comparar estados como conjuntos para reconocer estados iguales aunque sus elementos tengan distinto orden.	López Osorio Haniel
5	Interfaz gráfica	Incorporar funcionalidad a la GUI existente: cargar gramática, ejecutar el cálculo, visualizar la gramática y cada estado con sus elementos.	Ramírez Manzano Jaret
6	Pruebas y verificación de resultados	Preparar y comprobar las pruebas `g1`, `g2` y `g3` de la rúbrica. Contrastar elementos y transiciones con resultados calculados manualmente.	Soriano Martínez Miguel Ángel
7	UML e informe	Actualizar el UML conforme a la implementación real. Reunir explicaciones y evidencias.	Martínez Cesar Zaid

12 Glosario

Término	Definición
Analizador léxico	Programa que lee el texto fuente y lo divide en unidades llamadas tokens.
Token	Unidad léxica con significado propio (palabra reservada, identificador, número, operador, etc.).
Lexema	Cadena concreta de caracteres que corresponde a un token.
Tabla de símbolos	Estructura que registra los identificadores encontrados durante el análisis.
Flex	Herramienta que genera analizadores léxicos en C a partir de un archivo de reglas `.l`.
Gramática	Conjunto de reglas que describe la estructura válida de un lenguaje.
PRIMEROS (FIRST)	Conjunto de terminales con los que puede comenzar una cadena derivada de un símbolo.
SIGUIENTES (FOLLOW)	Conjunto de terminales que pueden aparecer inmediatamente después de un símbolo.
Colección canónica LR(0)	Conjunto de estados del autómata de ítems LR(0) que reconoce las producciones válidas de la gramática aumentada.

Término	Definición
Cerradura	Operación que, dado un conjunto de ítems LR(0), agrega las producciones de los no terminales que aparecen justo después del punto.
Ir_a (goto)	Operación que, dado un conjunto de ítems y un símbolo, desplaza el punto sobre ese símbolo y aplica la cerradura.
Estado de aceptación	Estado de la colección canónica alcanzado por el símbolo `$` (fin de la entrada).
TSV	Formato de texto de tabla con valores separados por tabulaciones.
venv	Entorno virtual de Python que aísla las dependencias del proyecto.
MSYS2	Entorno para Windows que ofrece herramientas de desarrollo como Flex y GCC mediante el gestor `pacman`.
PATH	Lista de carpetas donde el sistema busca los programas que se escriben en la terminal (`python`, `flex`, `gcc`).

Anexo A. Instalación manual en Windows

Usar este anexo solo si el instalador automático (sección 4.1) no funciona, por ejemplo cuando el equipo no tiene `winget`.

A.1 Instalar Python

Entrar a `python.org/downloads`, pulsar el botón **Download Python** y ejecutar el instalador descargado. En la primera pantalla, marcar la casilla **"Add python.exe to PATH"** antes de pulsar **Install Now**.

A.2 Instalar MSYS2, Flex y GCC

Entrar a `msys2.org`, descargar el instalador e instalarlo en la ruta por defecto (`C:\msys64`). Al terminar, abrir **MSYS2 UCRT64** desde el menú Inicio y ejecutar:

```
pacman -Syu
pacman -S --needed m4 mingw-w64-ucrt-x86_64-toolchain mingw-w64-ucrt-x86_64-flex
```

> **Nota:** si tras `pacman -Syu` la ventana se cierra sola, es normal: volver a abrir MSYS2 UCRT64 y repetir el comando antes de continuar.

A.3 Crear el entorno e instalar dependencias

Abrir PowerShell en la carpeta `INSTALL/` y ejecutar:

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

A.4 Abrir la aplicación

Con el entorno creado, usar `ejecutar.bat` como en la sección 4.1 (paso 3).